

## How the Code Works

This project implements preemptive multithreading with semaphore-based synchronization on the 8051 microcontroller. Unlike cooperative multithreading, the threads do not need to call `ThreadYield()` to switch execution. Instead, Timer 0 periodically interrupts the running thread and calls `myTimer0Handler()`, which saves the current thread state, selects the next runnable thread, restores its context, and returns using `RETI`.

The threading package stores each thread's saved stack pointer in internal RAM and uses a bitmap to track which thread IDs are active. Each thread has its own stack region and register bank. `Bootstrap()` initializes the thread system, creates `main()` as the first thread, enables the Timer 0 interrupt, starts Timer 0, and restores the context of the first thread.

This checkpoint uses a 3-deep bounded buffer shared by two producers and one consumer. The buffer is stored in three consecutive memory locations, and the program uses head and tail variables to manage it as a circular queue. Producer1 writes letters from A to Z repeatedly, while Producer2 writes digits from 0 to 9 repeatedly. The Consumer removes one character at a time from the buffer and sends it to the UART through SBUF.

The bounded buffer uses three semaphores: mutex, empty, and full. The mutex semaphore is initialized to 1 and ensures that only one thread can access the shared buffer at a time. The empty semaphore is initialized to 3 because the buffer starts with three empty slots. The full semaphore is initialized to 0 because the buffer initially contains no characters.

Before a producer inserts a character, it calls `SemaphoreWait(empty)` to make sure there is at least one empty slot, then calls `SemaphoreWait(mutex)` to enter the critical section. Inside the critical section, it writes the character into the buffer and updates the tail index. After leaving the critical section, it calls `SemaphoreSignal(mutex)` to release the buffer and `SemaphoreSignal(full)` to indicate that one more filled slot is available.

The Consumer calls `SemaphoreWait(full)` to make sure there is at least one character available, then calls `SemaphoreWait(mutex)` to enter the critical section. Inside the critical section, it reads a character from the buffer and updates the head index. After that, it calls `SemaphoreSignal(mutex)` to release the buffer and `SemaphoreSignal(empty)` to indicate that one more empty slot is available. The consumed character is then transmitted through the UART.

To improve fairness between the two producers, the program adds two extra semaphores: `letterTurn` and `digitTurn`. `letterTurn` is initialized to 1, while `digitTurn` is initialized to 0, so Producer1 starts first. After Producer1 produces one letter, it signals `digitTurn`, allowing

Producer2 to produce one digit. After Producer2 produces one digit, it signals letterTurn, allowing Producer1 to produce again. This forces the two producers to alternate instead of allowing one producer to dominate the buffer.

A fair output pattern is one where both producers make regular progress, such as A0B1C2D3E4.... This shows that the letter producer and digit producer are both being scheduled and both are successfully inserting characters into the bounded buffer. Since each producer must give the other producer a turn after producing one character, the program does not have producer starvation.

## 3.1 Screenshots for compilation

```
PS C:\Users\Theresia\Downloads\112006221_ppc4> make clean
powershell -NoProfile -Command "Remove-Item -ErrorAction SilentlyContinue *.h
ex,*.ihx,*.lnk,*.lst,*.map,*.mem,*.rel,*.rst,*.sym,*.asm,*.lk"
PS C:\Users\Theresia\Downloads\112006221_ppc4> make
sdcc -c test3threads.c
sdcc -c preemptive.c
sdcc -o test3threads.hex test3threads.rel preemptive.rel
PS C:\Users\Theresia\Downloads\112006221_ppc4> Get-ChildItem
```

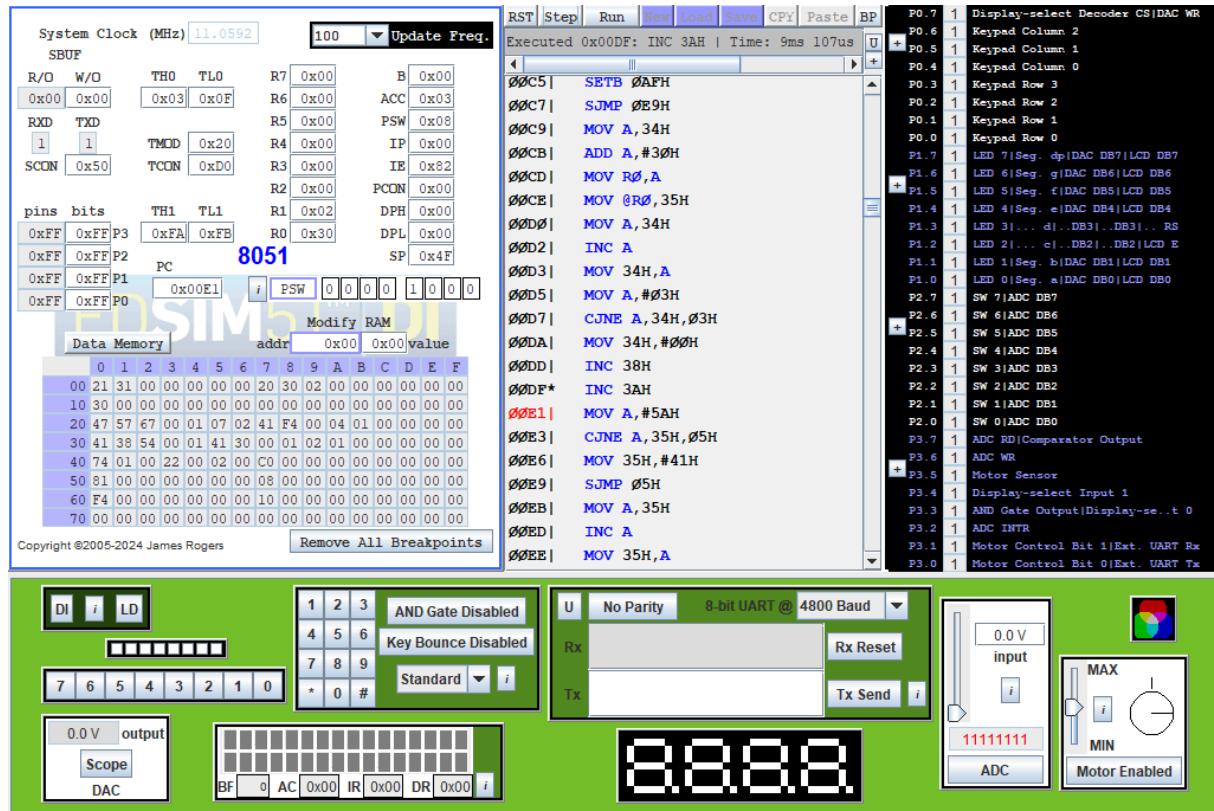
Directory: C:\Users\Theresia\Downloads\112006221\_ppc4

| Mode   | LastWriteTime     | Length | Name             |
|--------|-------------------|--------|------------------|
| -a---- | 6/13/2026 8:34 AM | 705    | Makefile         |
| -a---- | 6/13/2026 8:34 AM | 14639  | preemptive.asm   |
| -a---- | 6/13/2026 6:58 AM | 7396   | preemptive.c     |
| -a---- | 6/13/2026 8:00 AM | 1293   | preemptive.h     |
| -a---- | 6/13/2026 8:34 AM | 40145  | preemptive.lst   |
| -a---- | 6/13/2026 8:34 AM | 6265   | preemptive.rel   |
| -a---- | 6/13/2026 8:34 AM | 40799  | preemptive.rst   |
| -a---- | 6/13/2026 8:34 AM | 40876  | preemptive.sym   |
| -a---- | 6/13/2026 8:34 AM | 18520  | test3threads.asm |
| -a---- | 6/13/2026 8:33 AM | 4995   | test3threads.c   |
| -a---- | 6/13/2026 8:34 AM | 2379   | test3threads.hex |
| -a---- | 6/13/2026 8:34 AM | 263    | test3threads.lk  |
| -a---- | 6/13/2026 8:34 AM | 49213  | test3threads.lst |
| -a---- | 6/13/2026 8:34 AM | 17934  | test3threads.map |
| -a---- | 6/13/2026 8:34 AM | 1192   | test3threads.mem |
| -a---- | 6/13/2026 8:34 AM | 8010   | test3threads.rel |
| -a---- | 6/13/2026 8:34 AM | 50000  | test3threads.rst |
| -a---- | 6/13/2026 8:34 AM | 41117  | test3threads.sym |

```
PS C:\Users\Theresia\Downloads\112006221_ppc4>
```

## 3.2

### Part 1: When the Producer1 is running



Producer1 is running because the program counter is inside the Producer1() function and currentThread at address 0x24 is 0x01, meaning thread 1 is active. The screenshot was taken after Producer1 executed SemaphoreSignal(full), shown by the instruction INC 3AH. The semaphore full is stored at address 0x3A, so this instruction increases the number of filled buffer slots. The semaphore values show that mutex at 0x38 is 0x01, meaning the critical section has been released; empty at 0x39 is 0x02, meaning one empty slot was used; and full at 0x3A is 0x01, meaning one buffer slot now contains data. This confirms that Producer1 inserted a character into the bounded buffer.

## Part 2: When the Producer2 is running

The screenshot displays the Proteus 8.09 SP3 IDE with the following components:

- Register Window:** Shows the state of registers R0-R7, B, ACC, PSW, IP, IE, PCON, DPH, DPL, and SP. The Program Counter (PC) is highlighted at address 0x0154 with value 8051.
- Disassembly Window:** Shows the assembly code being executed. The current instruction is `INC 3AH` at address 0x152.
- Memory Window:** Shows the memory contents at address 0x00. The value at address 0x3A is 0x01, indicating the semaphore full flag.
- Hardware Simulation Window:** Shows the physical components of the system, including a DAC, a scope, a keyboard, a display, and a motor.

Producer2 is running because the program counter is inside the `Producer2()` function and `currentThread` at address 0x24 is 0x02, meaning thread 2 is active. The screenshot was taken after `Producer2` executed `SemaphoreSignal(full)`, shown by the instruction `INC 3AH`. The semaphore full is stored at address 0x3A, so this instruction increases the number of filled buffer slots. The semaphore values show that mutex at 0x38 is 0x01, meaning the critical section has been released; empty at 0x39 decreases because one empty buffer slot was used; and full at 0x3A increases because one buffer slot now contains data. This confirms that `Producer2` inserted a digit into the bounded buffer.

## Part 3: When the Consumer is running

The screenshot shows the Proteus ISIS simulation environment. The CPU window displays the assembly code for the Consumer thread, with the instruction `INC 39H` highlighted. The PC register shows the address `8051`. The I/O window shows the UART output as `11111111` and the DAC output as `0.0 V`. The keyboard window shows the '7' key pressed. The data memory window shows the state of the memory, with the semaphore at address `0x3A` reduced to `0x01`.

The Consumer is running because the program counter is inside the Consumer() function and currentThread at address 0x24 is 0x00, meaning thread 0 is active. The screenshot was taken after the Consumer executed SemaphoreSignal(empty), shown by the instruction INC 39H. Since the empty semaphore is stored at address 0x39, this shows that one buffer slot became available after the Consumer removed a character. The mutex semaphore at address 0x38 is 0x01, meaning the critical section has been released. The full semaphore at address 0x3A is reduced because there is one fewer filled buffer slot. This confirms that the Consumer removed a character from the bounded buffer and updated the semaphores correctly.

## Part 4: Fair-version UART

The screenshot displays the dsim5 IDE interface, which is used for simulating a microcontroller system. The top section shows the assembly code being executed, with instructions like `ORG 0000H`, `LJMP 01EFH`, `RETI`, `ORG 000BH`, `LJMP 01F6H`, `ORG 0010H`, `NOP`, `MOVX A, @R0`, `MOV R3, A`, `MOV A, R2`, `MOVX @R0, A`, `SJMP 2CH`, `NOP`, `MOVX A, @DPTR`, `MOV R3, A`, `MOV A, R2`, `MOVX @DPTR, A`, `SJMP 24H`, and `MOV A, @R0`. The registers section shows the current state of various registers, including R0 through R7, B, ACC, PSW, IE, PCON, DPH, DPL, SP, and PC. The PC register is highlighted with the value 8051. The memory section shows a data memory table with addresses from 00 to 70 and their corresponding values. The bottom section shows the hardware peripherals, including a keypad, a display, an ADC, and a motor. The UART output is shown as a sequence of characters: A0B1C2D3E4F5G6H7I8J9K0L1M2....

The UART output of the fair version shows an alternating pattern between letters and digits, such as A0B1C2D3E4F5G6H7I8J9K0L1M2.... This indicates that both Producer1 and Producer2 are making regular progress. Unlike the unfair version, where one producer may dominate the output, the fair version forces the two producers to alternate using the additional semaphores letterTurn and digitTurn. Therefore, the output demonstrates that the scheduling behavior is fair and that starvation is avoided.

## Part 5: Unfair-version UART

The screenshot displays the DSIM5 IDE interface, which is used for simulating a microcontroller system. The top section shows the assembly code being executed, with the current instruction being `MOVX A, @R0` at address `00201`. The registers section shows the current state of the microcontroller's registers, including the Program Counter (PC) at `0x0084` and the Stack Pointer (SP) at `0x4F`. The memory section shows the current state of the microcontroller's memory, with the PC value of `8051` highlighted. The bottom section shows a hardware simulation interface with various components including a display, a keypad, a motor, and a DAC. The display shows the number `8888`, and the motor is currently disabled.

The UART output of the unfair version shows mostly letters without alternating digits. This means Producer1 is making progress repeatedly, while Producer2 rarely or never gets a chance to insert characters into the bounded buffer. Since the output is dominated by one producer, this behavior is not fair. It also indicates that starvation can occur in the unfair version, because one producer may be prevented from making regular progress.